

INFO-F-302, Cours d'Informatique Fondamentale

Emmanuel Filiot et Jean-François Raskin

Département d'Informatique

Faculté des Sciences

Université Libre de Bruxelles

Année académique 2026-2027

Aux origines, la logique

- ▶ **Théorème d'incomplétude de Gödel (1931)** : il existe des énoncés¹ valides qui ne sont pas démontrables.

1. écrits en logique des prédicats

Aux origines, la logique

- ▶ **Théorème d'incomplétude de Gödel (1931)** : il existe des énoncés¹ valides qui ne sont pas démontrables.
- ▶ Gödel avait posé une question : **peut-on automatiser les mathématiques ?**
Est-ce qu'il existe un procédé automatique (un algorithme) qui, étant donnée un énoncé, permet de décider s'il est vrai ou non ?

1. écrits en logique des prédicats

Aux origines, la logique

- ▶ **Théorème d'incomplétude de Gödel (1931)** : il existe des énoncés¹ valides qui ne sont pas démontrables.
- ▶ Gödel avait posé une question : **peut-on automatiser les mathématiques ?**
Est-ce qu'il existe un procédé automatique (un algorithme) qui, étant donnée un énoncé, permet de décider s'il est vrai ou non ?
- ▶ Turing y répondit négativement (1936). En particulier :
 - ▶ il a défini la notion d'algorithme par un modèle abstrait d'ordinateur (machines de Turing)
 - ▶ il a démontré qu'il n'est pas possible de décider algorithmiquement qu'une machine de Turing termine ou non (problème de l'arrêt)
 - ▶ la propriété P_i = "la i -ème machine s'arrête" est définissable en logique (pour une certaine "numérotation" des machines).
 - ▶ ceci répond à la question de Gödel : on ne peut prouver la propriété P_i (ou sa négation) algorithmiquement.

1. écrits en logique des prédicats

Théorie de la complexité

- ▶ Turing a démontré que certains problèmes (le problème de l'arrêt) sont indécidables
- ▶ lorsqu'un problème est décidable algorithmiquement, on souhaiterait en savoir plus : combien d'opérations sont nécessaires intrinsiquèment ? avec quelle mémoire ?
- ▶ ceci a donné naissance à la théorie de la complexité : classifier les problème selon leur complexité algorithmique intrinsèque (en temps et en mémoire)

Informatique Fondamentale

L'objectif de ce cours est de vous fournir les ingrédients de base qui vous permettront de comprendre de manière plus précise ce qui a été dit précédemment, et de répondre aux questions (soit lors de ce cours, soit dans le cours calculabilité et complexité de master) :

- ▶ qu'est-ce que la logique ?
- ▶ qu'est-ce qu'un énoncé vraie ?
- ▶ qu'est-ce qu'une preuve ?
- ▶ qu'est-ce qu'un problème indécidable ? comment peut-on le démontrer ?
- ▶ qu'est-ce qu'une classe de complexité ? qu'est-ce qu'un problème complet pour une classes ?

Informatique Fondamentale

L'objectif de ce cours est de vous fournir les ingrédients de base qui vous permettront de comprendre de manière plus précise ce qui a été dit précédemment, et de répondre aux questions (soit lors de ce cours, soit dans le cours calculabilité et complexité de master) :

- ▶ qu'est-ce que la logique ?
- ▶ qu'est-ce qu'un énoncé vraie ?
- ▶ qu'est-ce qu'une preuve ?
- ▶ qu'est-ce qu'un problème indécidable ? comment peut-on le démontrer ?
- ▶ qu'est-ce qu'une classe de complexité ? qu'est-ce qu'un problème complet pour une classes ?

Informatique fondamentale et modélisation

L'informatique fondamentale est une branche de l'informatique dont le principal but est la conception de **modèles** pour des problèmes, concepts ou applications informatiques, et le développement **d'algorithmes** pour analyser ces modèles.

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.
- ▶ les systèmes de déduction : modélisent le raisonnement mathématique via la notion de preuve

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.
- ▶ les systèmes de déduction : modélisent le raisonnement mathématique via la notion de preuve
- ▶ les machines de Turing : modèle de calcul

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.
- ▶ les systèmes de déduction : modélisent le raisonnement mathématique via la notion de preuve
- ▶ les machines de Turing : modèle de calcul
- ▶ les automates : modélisent des programmes informatiques “simples”

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.
- ▶ les systèmes de déduction : modélisent le raisonnement mathématique via la notion de preuve
- ▶ les machines de Turing : modèle de calcul
- ▶ les automates : modélisent des programmes informatiques “simples”
- ▶ chaînes de Markov : modélisent des systèmes probabilistes

Exemples de modèles

- ▶ graphes : modèles de réseaux – sociaux, routiers, web, etc. –, de dépendances (entre les variables d'un programme, entre les classes en programmation objet, etc.), ...
- ▶ la logique : modélise les énoncés mathématiques, les propriétés d'un système informatique, d'un programme, etc.
- ▶ les systèmes de déduction : modélisent le raisonnement mathématique via la notion de preuve
- ▶ les machines de Turing : modèle de calcul
- ▶ les automates : modélisent des programmes informatiques “simples”
- ▶ chaînes de Markov : modélisent des systèmes probabilistes
- ▶ etc.

Une notion centrale du cours : réductions entre problèmes

- ▶ une notion essentielle en informatique fondamentale
- ▶ modélisation d'un problème par un autre

Pour les problèmes de décision (réponse oui/non)

Réduire un problème A vers un problème B , c'est trouver une méthode permettant d'encoder toute entrée I_A du problème A comme une entrée I_B du problème B , telle que

I_A a une solution (la réponse est oui) si et seulement si I_B a une solution

- ▶ parfois on demandera que cette méthode soit un algorithme
- ▶ pour les problèmes qui ne sont pas des problèmes de décision (trier un tableau par exemple), on demandera que toute solution de I_A s'encode en une solution de I_B et réciproquement toute solution de I_B se décode en une solution de I_A

Exemple – Problème B : Bin Packing

Problème Bin Packing (B) :

- ▶ **INPUT** : n objets de poids $p_1, \dots, p_n$, k sacs de capacité C
- ▶ **OUTPUT** : oui ssi on peut ranger tous les objets dans au plus k sacs sans dépasser la capacité de chaque sac

Exemple – Problème B : Bin Packing

Problème Bin Packing (B) :

- **INPUT** : n objets de poids $p_1, \dots, p_n$, k sacs de capacité C
- **OUTPUT** : oui ssi on peut ranger tous les objets dans au plus k sacs sans dépasser la capacité de chaque sac

On veut ranger les objets suivants dans des sacs de capacité 10kg.

Objets	1	2	3	4	5	6	7
Poids	3	4	4	3	3	2	1

Peut-on réussir avec 3 sacs ?

Exemple – Problème B : Bin Packing

Problème Bin Packing (B) :

- ▶ **INPUT** : n objets de poids $p_1, \dots, p_n$, k sacs de capacité C
- ▶ **OUTPUT** : oui ssi on peut ranger tous les objets dans au plus k sacs sans dépasser la capacité de chaque sac

On veut ranger les objets suivants dans des sacs de capacité 10kg.

Objets	1	2	3	4	5	6	7
Poids	3	4	4	3	3	2	1

Peut-on réussir avec 3 sacs ? oui, on forme les sacs d'objets suivants : $\{1, 2\}$, $\{3, 4, 5\}$, $\{6, 7\}$.

Exemple – Problème B : Bin Packing

Problème Bin Packing (B) :

- **INPUT** : n objets de poids $p_1, \dots, p_n$, k sacs de capacité C
- **OUTPUT** : oui ssi on peut ranger tous les objets dans au plus k sacs sans dépasser la capacité de chaque sac

On veut ranger les objets suivants dans des sacs de capacité 10kg.

Objets	1	2	3	4	5	6	7
Poids	3	4	4	3	3	2	1

Peut-on réussir avec 3 sacs ? oui, on forme les sacs d'objets suivants :
 $\{1, 2\}$, $\{3, 4, 5\}$, $\{6, 7\}$.
 et avec 2 sacs ?

Exemple – Problème B : Bin Packing

Problème Bin Packing (B) :

- ▶ **INPUT** : n objets de poids $p_1, \dots, p_n$, k sacs de capacité C
- ▶ **OUTPUT** : oui ssi on peut ranger tous les objets dans au plus k sacs sans dépasser la capacité de chaque sac

On veut ranger les objets suivants dans des sacs de capacité 10kg.

Objets	1	2	3	4	5	6	7
Poids	3	4	4	3	3	2	1

Peut-on réussir avec 3 sacs ? oui, on forme les sacs d'objets suivants : $\{1, 2\}, \{3, 4, 5\}, \{6, 7\}$.

et avec 2 sacs ?

oui : $\{1, 2, 4\}, \{3, 5, 6, 7\}$. Cette solution est optimale (on ne peut pas réussir avec un seul sac puisque la somme totale de tous les poids dépasse 10).

Problème A : 2-partition

Problème 2-partition (A) :

- ▶ **INPUT** : m entiers naturels $c_1, \dots, c_m$ tels que $S := \sum_{i=1}^m c_i$ est paire,
- ▶ **OUTPUT** : oui ssi il existe $J \subseteq \{1, \dots, m\}$ tel que $\sum_{i \in J} c_i = \sum_{i \notin J} c_i$.

Problème A : 2-partition

Problème 2-partition (A) :

- ▶ **INPUT** : m entiers naturels $c_1, \dots, c_m$ tels que $S := \sum_{i=1}^m c_i$ est paire,
- ▶ **OUTPUT** : oui ssi il existe $J \subseteq \{1, \dots, m\}$ tel que
$$\sum_{i \in J} c_i = \sum_{i \notin J} c_i.$$

Exemple avec 1, 4, 2, 3, 2 : $1 + 2 + 3 = 2 + 4$.

Problème A : 2-partition

Problème 2-partition (A) :

- ▶ **INPUT** : m entiers naturels $c_1, \dots, c_m$ tels que $S := \sum_{i=1}^m c_i$ est paire,
- ▶ **OUTPUT** : oui ssi il existe $J \subseteq \{1, \dots, m\}$ tel que
$$\sum_{i \in J} c_i = \sum_{i \notin J} c_i.$$

Exemple avec $1, 4, 2, 3, 2$: $1 + 2 + 3 = 2 + 4$.

Réduction vers BinPacking ?

Problème A : 2-partition

Problème 2-partition (A) :

- **INPUT** : m entiers naturels $c_1, \dots, c_m$ tels que $S := \sum_{i=1}^m c_i$ est paire,
- **OUTPUT** : oui ssi il existe $J \subseteq \{1, \dots, m\}$ tel que
$$\sum_{i \in J} c_i = \sum_{i \notin J} c_i.$$

Exemple avec $1, 4, 2, 3, 2$: $1 + 2 + 3 = 2 + 4$.

Réduction vers BinPacking ? On prend $C = S/2$ et les objets $1, \dots, m$ de poids $c_1, \dots, c_m$, et $k = 2$. Il y a une solution à toute instance de 2-partition si et seulement si il y a en une à l'instance de BinPacking ainsi construite.

Problème A : 2-partition

Problème 2-partition (A) :

- ▶ **INPUT** : m entiers naturels $c_1, \dots, c_m$ tels que $S := \sum_{i=1}^m c_i$ est paire,
- ▶ **OUTPUT** : oui ssi il existe $J \subseteq \{1, \dots, m\}$ tel que
$$\sum_{i \in J} c_i = \sum_{i \notin J} c_i.$$

Exemple avec $1, 4, 2, 3, 2$: $1 + 2 + 3 = 2 + 4$.

Réduction vers BinPacking ? On prend $C = S/2$ et les objets $1, \dots, m$ de poids $c_1, \dots, c_m$, et $k = 2$. Il y a une solution à toute instance de 2-partition si et seulement si il y a en une à l'instance de BinPacking ainsi construite.

Remarque : si nous disposons d'un algorithme pour résoudre le problème BinPacking, alors on peut l'utiliser pour résoudre le problème 2-partition !

9 Objectifs du cours

Être capable

- ▶ de modéliser des problèmes de manière précise et rigoureuse en utilisant des langages logiques : logique Booléenne et logique des prédicats
- ▶ utiliser des solveurs logique pour résoudre des problèmes
- ▶ de faire du raisonnement déductif en logique
- ▶ de construire et d'*analyser* des programmes simples représentables par automates

9 Objectifs du cours

Être capable

- ▶ de modéliser des problèmes de manière précise et rigoureuse en utilisant des langages logiques : logique Booléenne et logique des prédicats
- ▶ utiliser des solveurs logique pour résoudre des problèmes
- ▶ de faire du raisonnement déductif en logique
- ▶ de construire et d'*analyser* des programmes simples représentables par automates

+ vous préparer au master d'informatique (cours de calculabilité et complexité notamment)

10 Plan

1. Partie 1 (~12h, Emmanuel Filiot) : logique Booléenne
 - 1.1 syntaxe, sémantique, satisfaisabilité, validité
 - 1.2 déduction naturelle
 - 1.3 SAT solveurs
2. Partie 2 (~28h, Jean-François Raskin) : logique des prédicats et automates
 - 2.1 syntaxe, sémantique
 - 2.2 déduction naturelle
 - 2.3 résolution
 - 2.4 automates finis et expressions rationnelles
3. Partie 3 (~4h, Jean-François Raskin) : sujets avancés (limite algorithmique pour le raisonnement logique)

11 Organisation pratique du cours

Annonces Par email et reprises sur l'UV.

Matériel Les slides constituent le syllabus. Ils seront disponibles sur l'UV.

Travaux pratiques (2 groupes) Assistants : Arnaud Leponce et Nathan Vanzeir

Evaluation Examen écrit.

Contact ► **email** : `efiliot@ulb.be` et
`jean-francois.raskin@ulb.be`